AI Resume Scorer

INTRODUCTION:

AI Resume Scorer is a full-stack, production-grade resume intelligence platform. Candidates upload resumes, match them against job descriptions, identify skill gaps, and track their analysis history all through a polished, responsive dashboard powered by a streaming microservices pipeline. The platform is built on the MERN stack (MongoDB, Express.js, React, Node.js) with TypeScript throughout. It integrates Apache Kafka for asynchronous event-driven communication between microservices, Redis for caching, and the Nebius AI API (Llama-3.3-70B) for deep semantic analysis of resumes and job descriptions.AI Resume Scorer is designed to help job seekers understand exactly how well their resume matches a target role, what skills they are missing, and how to improve their resume all through an intelligent, automated pipeline that processes resumes end-to-end from upload to scored, enhanced output.

SCENARIO Based Case Study:

Meet Priya - A Job Seeker Navigating the Modern Hiring Market

Priya is a software engineering graduate with two years of work experience who is actively applying for mid-level developer roles at technology companies. She finds the job application process overwhelming .she is unsure whether her resume is tailored for each job description, whether she is missing critical skills that ATS systems flag, and why she rarely gets callbacks despite applying to many positions.

A friend introduces Priya to AI Resume Scorer. Intrigued by the promise of data-driven feedback, she decides to explore how the platform can help her land more interviews.

- User Registration and Authentication: Priya registers an account on AI Resume Scorer, providing her email and setting a secure password. She logs in using JWT-based authentication, and all her resume data remains private and protected.
- Resume Upload: Priya uploads her resume PDF through the drag-and-drop upload interface. The system stores her file and immediately triggers the processing pipeline via a Kafka event.
- AI Analysis Pipeline: Behind the scenes, a chain of microservices processes her resume — the parser extracts raw text, the embedder generates semantic vectors using Nebius AI, the skill extractor identifies her technical competencies, the matcher compares her profile to the job description, and the feedback service generates actionable recommendations.
- Match Score and Skill Gap Report: Priya views her match percentage against a target job description on her dashboard. She can clearly see which skills she possesses and which ones she is missing — giving her a concrete action plan.

- Resume Enhancement: Priya clicks 'Improve Resume.' The AI enhancer upgrades her bullet points with quantified achievements, fills in missing sections, and produces a professionally typeset one-page PDF that she can download and submit.
- Order History and Analytics: Priya can view all past resume analyses, track her improvement over time, and delete individual entries or clear her entire history.

Thanks to AI Resume Scorer, Priya now approaches job applications with confidence. She understands exactly how her resume stacks up against each job description, knows which skills to develop next, and has a polished, ATS-optimized resume ready to submit. The platform has become an essential part of her job search toolkit.

SYSTEM REQUIREMENTS:

Software Requirements:

The following tools and platforms are required to develop, test, and run the AI Resume Scorer application:

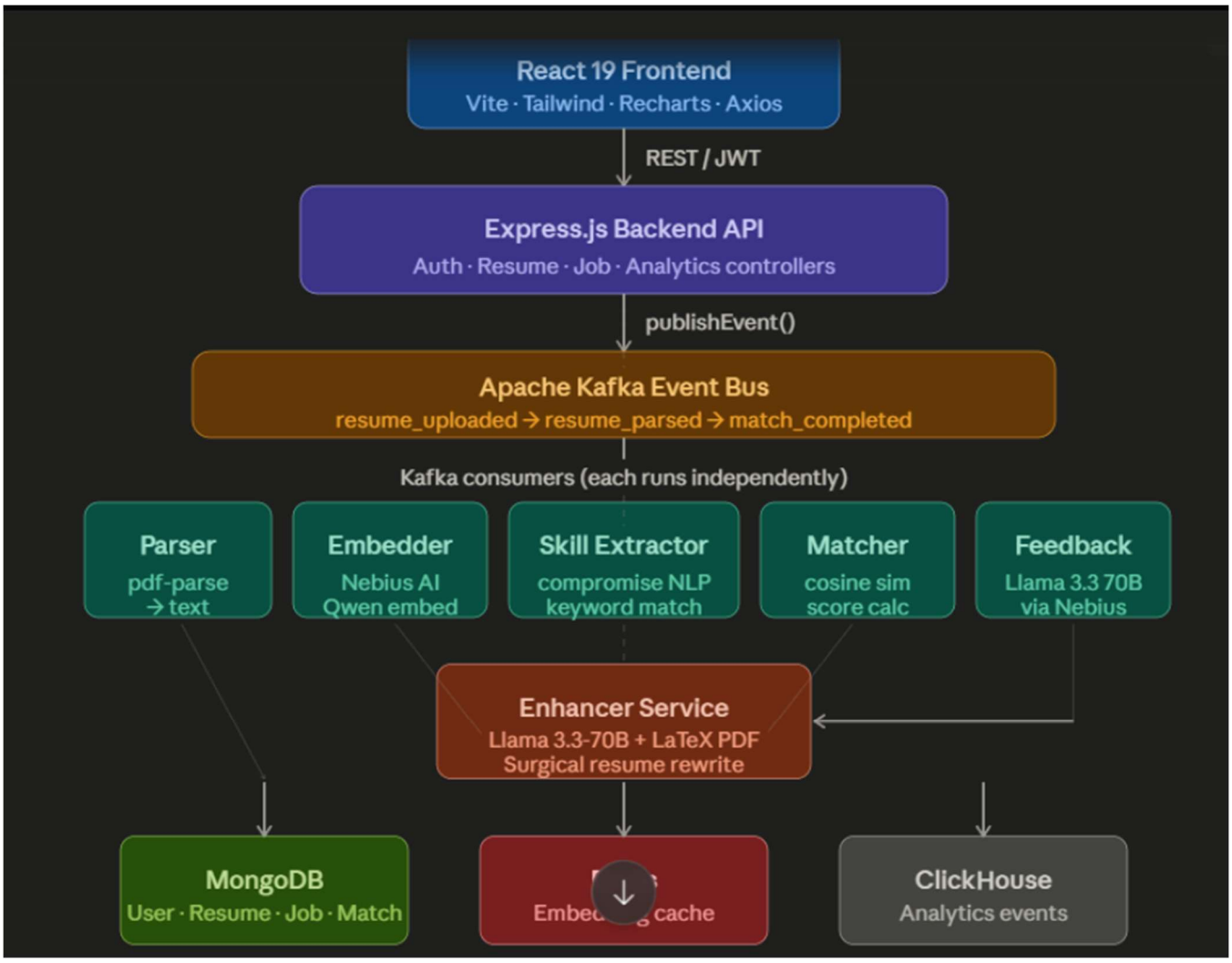
- **Operating System:** Windows 10/11, macOS, or Linux — supports cross-platform development and testing
- **Node.js (v18 or above):** Provides the runtime environment for the backend API, all microservices, and the frontend build toolchain
- **npm (v8 or above):** Package manager required to install all dependencies across the monorepo
- **React 19 (via Vite):** A JavaScript library for building the dynamic, responsive user interface
- **TypeScript 5.x:** Provides static typing across both the frontend and backend for safer, more maintainable code
- **Express.js:** A lightweight web framework for building the RESTful API backend
- **MongoDB:** A NoSQL database used to store users, resumes, jobs, skills, and match results
- **Redis:** An in-memory data store used for caching and session management
- **Apache Kafka (KafkaJS):** A distributed event streaming platform that powers the asynchronous microservice pipeline
- **Docker / docker-compose:** Used for containerized local deployment of Kafka, Redis, and MongoDB
- **Postman:** Tool for testing and validating REST API endpoints during development
- **Visual Studio Code:** Preferred code editor with built-in Git, terminal, and TypeScript support
- **Browser:** Google Chrome or Firefox (latest version) for rendering and testing the frontend UI

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — required for running multiple concurrent development servers and services
- **RAM:** Minimum 8 GB (16 GB recommended) — running the backend, frontend, Kafka broker, Redis, and multiple microservices simultaneously requires significant memory
- **Storage:** At least 2 GB free space — required for package installations, Docker images, MongoDB data, and local file uploads
- **Display:** 1366x768 or higher — recommended for an optimal coding and dashboard visualization experience

PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:



AI Resume Scorer is built on an event-driven microservices architecture. The frontend communicates exclusively with the backend REST API. The backend, upon receiving a resume upload, publishes a

Kafka event that triggers a sequential pipeline of independent microservices. Each service does one job — parsing, embedding, skill extraction, matching, or feedback — and passes the result to the next via Kafka topics. This ensures the system is highly scalable, resilient, and asynchronous.

Layer	Technologies
Frontend	React 19, Vite, TypeScript, React Router, Axios, Tailwind CSS, Recharts, Framer Motion, React Toastify
Backend API	Node.js, Express.js, TypeScript, Mongoose, JWT, bcryptjs, multer, express-rate-limit
Data Layer	MongoDB (primary database), Redis (cache / rate limiting)
Message Broker	Apache Kafka (KafkaJS) — event-driven pipeline backbone
Microservices	Parser · Embedder · Skill Extractor · Matcher · Feedback · Enhancer
AI / ML	Nebius AI — Llama-3.3-70B for semantic embeddings, analysis, and enhancement
Deployment	PM2 (process management), Nginx (reverse proxy), Docker / docker-compose

Event-Driven Pipeline(Kafka)

The core intelligence of the platform flows through a Kafka-powered pipeline. The sequence of events after a resume is uploaded is as follows:

- **resume_uploaded** → Backend publishes event to Kafka after file is saved to storage
- **resume_parsed** → Parser service extracts raw text from the PDF using pdf-parse
- **resume_embedded** → Embedder service converts text to semantic vectors via Nebius AI
- **skills_extracted** → Skill extractor identifies technical competencies from parsed text
- **resume_matched** → Matcher service calculates cosine similarity score between resume and job description vectors
- **feedback_generated** → Feedback service produces natural language recommendations based on the skill gap

ER DIAGRAM:

The Entity-Relationship (ER) diagram for AI Resume Scorer represents the core data entities and their relationships within the system. Below is a description of the key components:

1. Entities:

- **User:** Represents registered users of the platform. Attributes: userId (ObjectId), name (String), email (String, unique), password (String, hashed), avatarUrl (String), timestamps.

- **Resume:** Represents uploaded resumes. Attributes: resumeId (ObjectId), userId (ref: User), fileUrl (String), fileName (String), parsedText (String), resumeVector ([Number]), feedback (String), improvedResumeUrl (String), status (Enum: UPLOADED → PARSING → PARSED → EMBEDDING → EMBEDDED → SKILL_EXTRACTED → COMPLETED / FAILED), timestamps.
- **Job:** Represents job descriptions entered by users. Attributes: jobId (ObjectId), title (String), company (String), description (String), jobVector ([Number]), timestamps.
- **Skill:** Represents individual skills parsed from resumes or job descriptions. Attributes: skillId (ObjectId), name (String, unique), category (String).
- **MatchResult:** Represents the outcome of matching a resume to a job. Attributes: matchId (ObjectId), resumeId (ref: Resume), jobId (ref: Job), score (Number), matchPercentage (Number), skillGap ([String]), confidence (Number), basis (String), timestamps.

2.Relationships

- User → Resume: One-to-many. A user can upload multiple resumes; each resume belongs to one user.
- Resume → Skill: Many-to-many. A resume can reference many skills; a skill can appear in many resumes.
- Job → Skill: Many-to-many. A job description can require many skills; a skill can appear in many job listings.
- Resume → MatchResult: One-to-many. A resume can be matched against multiple jobs, each producing a MatchResult.
- Job → MatchResult: One-to-many. A job can be matched against multiple resumes, each producing a MatchResult.

3.Primary Keys and Foreign Keys

- Each entity uses a MongoDB ObjectId as its primary key (_id).
- Resume references User via userId (foreign key).
- MatchResult references both Resume (resumeId) and Job (jobId) as foreign keys.
- A unique compound index is enforced on (resumeId, jobId) in MatchResult to prevent duplicate match entries.

Key Features:

- **User Registration and Authentication:** Secure sign-up and login flows with password hashing via bcryptjs and JWT issuance. All API routes are protected by authentication middleware.
- **Resume Upload and Management:** Drag-and-drop PDF upload interface via react-dropzone. Full CRUD for resumes including individual delete and bulk history clear.

- **AI-Powered Semantic Matching:** Cosine similarity between Nebius AI-generated vectors of the resume and job description provides a precise, semantically meaningful match percentage — going beyond simple keyword matching.
- **Skill Gap Analysis:** The skill extractor identifies which competencies the candidate has and which ones the target job requires. The gap is presented clearly on the dashboard.
- **Actionable AI Feedback:** The feedback microservice generates natural language recommendations explaining how the candidate can improve their resume and close identified skill gaps.
- **Resume Enhancement with AI:** The enhancer service uses Llama-3.3-70B to surgically upgrade bullet points, inject quantified achievements, and produce a polished, professional one-page PDF styled like a LaTeX resume.
- **Dashboard and Analytics:** Interactive Recharts visualizations show match scores, skill distribution, and analysis history over time, helping users track their improvement.
- **ATS Optimization:** The platform helps users ensure their resumes include the right keywords and phrasing to pass Applicant Tracking System filters.
- **Resume History:** All previous analyses are stored per user. Users can revisit, compare, or delete past entries at any time.
- **Job Description Management:** Users can create, list, update, and delete job descriptions through the platform for repeated matching against different resume versions.

ROLES AND RESPONSIBILITIES:

User:

- Registration and Account Management: Candidates create accounts, provide their email and password, and manage their profile and account security.
- Resume Upload: Users upload their resume PDFs and trigger the AI analysis pipeline.
- Job Description Entry: Users enter or paste the job description they want to match their resume against.
- Viewing Results: Users view their match score, skill gap analysis, and AI-generated feedback on the analysis page.
- Resume Enhancement: Users can request an AI-enhanced version of their resume and download the generated PDF.
- History Management: Users can view all previous analyses, compare results over time, and delete entries they no longer need.

Admin:

- Pipeline Management: The backend orchestrates the entire event-driven pipeline, publishing and consuming Kafka events to coordinate microservices.
- Data Integrity: The system enforces unique constraints (e.g., unique email, unique resume-job match pairs) and validates input at the API layer.
- Security Enforcement: JWT middleware protects all data routes. Passwords are never stored in plaintext.
- Rate Limiting: express-rate-limit prevents abuse of the upload and analysis endpoints.
- Deployment Management: PM2 manages all processes in production; Nginx handles reverse proxying and SSL termination.

User Flow:

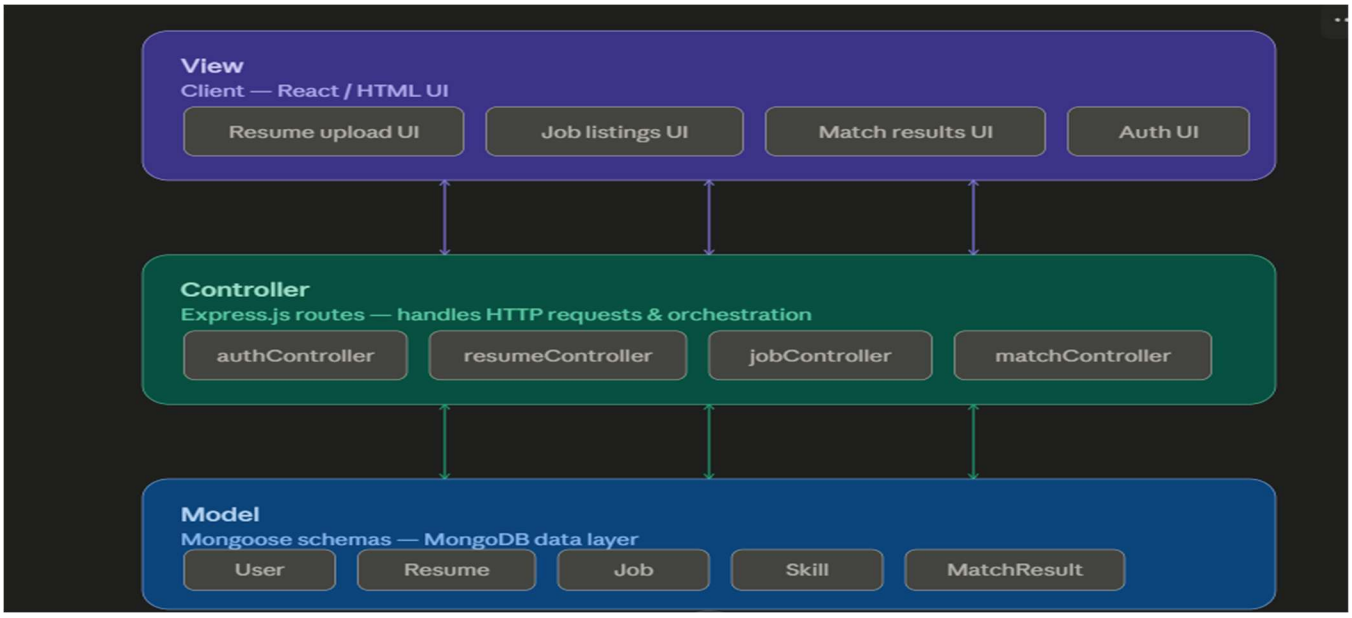
Candidate flow:

- Register / Login — Create a secure account or log in with existing credentials.
- Upload Resume — Drag and drop a PDF resume onto the upload interface.
- Enter Job Description — Paste or type the target job description.
- Wait for Analysis — The Kafka pipeline processes the resume through parsing, embedding, skill extraction, matching, and feedback.
- View Results — Review the match score, skill gap breakdown, and AI feedback on the analysis page.
- Enhance Resume — Optionally request an AI-improved resume PDF for download.
- Manage History — Browse past analyses, compare results, or delete entries.

System Pipeline Flow:

- Upload Received — Backend stores file and publishes to resume_uploaded Kafka topic.
- Parser Service — Consumes event, extracts raw text from PDF, updates status to PARSED.
- Embedder Service — Converts parsed text to semantic vectors using Nebius AI, updates status to EMBEDDED.
- Skill Extractor — Identifies skills from parsed text, updates status to SKILL_EXTRACTED.
- Matcher Service — Calculates cosine similarity against job vector, stores MatchResult.
- Feedback Service — Generates natural language recommendations, stores feedback text.
- Status → COMPLETED — Frontend polls or receives updated status; dashboard displays results.

MVC PATTERN:



The AI Resume Scorer backend follows the Model-View-Controller (MVC) architectural pattern, separating data logic, business logic, and routing into clearly defined layers. This separation promotes modularity, maintainability, and testability.

Model Layer (Data Layer)

The Model layer defines all data schemas and handles database operations using Mongoose. Each model corresponds to a MongoDB collection.

- User — Schema defining user accounts with email, hashed password, name, and avatarUrl.
- Resume — Schema tracking uploaded resumes including their processing status, parsed text, vector embeddings, and links to enhanced versions.
- Job — Schema for job descriptions including title, company, description text, and computed vector.
- Skill — Schema for individual skills with a unique name and category.
- MatchResult — Schema for match outcomes between a specific resume and job, storing score, matchPercentage, skillGap array, and confidence.

Controller Layer

Controllers contain all business logic. They receive validated requests from routes, interact with models, and return structured responses to the client.

- auth.controller.js — Handles user registration (signUp), login (signIn), and profile retrieval (me). Manages JWT creation and password hashing with bcryptjs.
- resume.controller.js — Handles resume upload (with multer), listing, updating, deletion, and bulk history clearing. Publishes Kafka events on upload.
- job.controller.js — Handles full CRUD for job descriptions and triggers job vector embedding.
- analytics.controller.js — Aggregates match data to produce skill gap insights and dashboard statistics.

View Layer (Routing Layer)

In the context of a REST API, the View is implemented as the routing layer. Each route file maps HTTP methods and paths to the corresponding controller functions.

- auth.routes.js — POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
- resume.routes.js — GET/POST/PUT/DELETE /api/resumes, DELETE /api/resumes/clear
- job.routes.js — GET/POST/PUT/DELETE /api/jobs

- analytics.routes.js — GET /api/analytics (match scores, skill gap data)

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features can be added easily by creating new routes, controllers, and models without disrupting existing code.
- Reusability: Business logic in controllers and models can be reused across multiple routes.
- Testability: Each layer can be tested independently, especially controllers and models.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflicts.

Project Setup And Configuration

● Creating project folder

- Create a root project folder (e.g., ai-resume-scorer).
- Inside, create three main folders: frontend, backend, and services.
- Inside services, create subfolders for each microservice: parser, embedder, skill-extractor, matcher, feedback, enhancer.
- Open the root folder in VS Code.

Frontend Setup(React with Vite)

  - Navigate to the frontend folder in the VS Code terminal.
  - Run: npm create vite@latest . -- --template react-ts
  - Select React as the framework and TypeScript as the variant.
  - Run: npm install to install all packages.
  - Install additional dependencies: npm install axios react-router-dom react-toastify recharts framer-motion lucide-react react-dropzone
  - Run: npm run dev to start the Vite development server.

Backend Setup

  - Navigate to the backend folder in the VS Code terminal.
  - Run: npm init -y to initialise the Node.js project.
  - Create the main entry file: server.js (or server.ts).
  - Create folders: controllers, models, routes, middleware, config.
  - Install dependencies: npm install express mongoose dotenv cors bcryptjs jsonwebtoken multer kafkajs ioredis express-rate-limit
  - Create a .env file with DATABASE_URL, JWT_SECRET, API_PORT, REDIS_URL, KAFKA_BROKERS.

- Run: npm run dev to start the backend server.

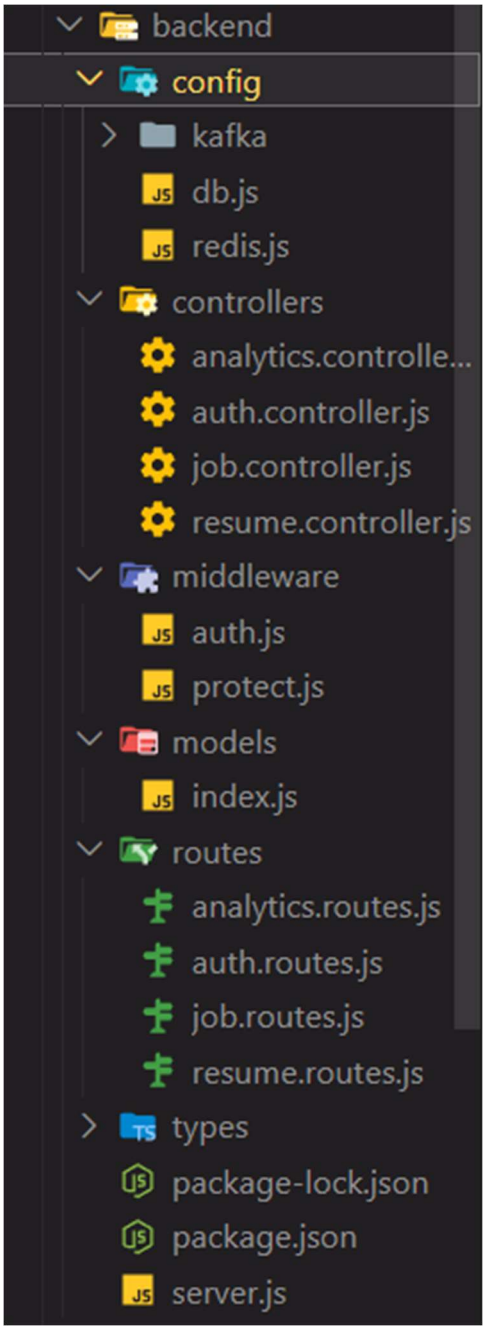
Microservice Setup

- Each microservice (parser, embedder, skill-extractor, matcher, feedback, enhancer) is an independent Node.js project.
- Navigate to each service folder and run npm init -y, then npm install the required dependencies.
- Each service connects to Kafka, consumes events from its input topic, processes the payload, and publishes to its output topic.
- Run all services concurrently from the root using: npm run dev (configured in the root package.json using concurrently).

BACKEND DEVELOPMENT:

Backend Reference Video:
Link

Backend Structure:



Controllers

- auth.controller.js — Handles user registration (signUp), login (signIn), and authenticated profile retrieval (me). Implements bcryptjs password hashing and JWT signing.
- resume.controller.js — Manages the full resume lifecycle: upload (multer), list, update, delete (individual and bulk). Publishes a Kafka event on every successful upload to trigger the analysis pipeline.
- job.controller.js — Full CRUD for job descriptions. On creation, triggers the embedding of the job description vector for future matching.

- analytics.controller.js — Aggregates MatchResult data to produce dashboard statistics including average match scores and skill frequency distribution.

Config

- db.js — MongoDB connection setup using Mongoose with connection caching for efficient reuse across serverless-style invocations.
- redis.js — Redis client setup using ioredis for caching and rate-limiting support.
- kafka/ — Kafka producer and consumer setup using KafkaJS. Includes topic definitions and connection management for both the backend API and microservices.

Middleware

- auth.js / protect.js — JWT validation middleware. Extracts and verifies the Bearer token from the Authorization header and attaches the decoded user to req.user.
- multer — File upload handling middleware. Stores uploaded resume PDFs to the local uploads/ directory.

Models

- User — name, email (unique), password (hashed), avatarUrl, timestamps.
- Resume — userId (ref: User), fileUrl, fileName, parsedText, resumeVector, feedback, improvedResumeUrl, status (Enum), skills (ref: Skill), timestamps.
- Job — title, company, description, jobVector, skills (ref: Skill), timestamps.
- Skill — name (unique), category.
- MatchResult — resumeId (ref: Resume), jobId (ref: Job), score, matchPercentage, skillGap, confidence, basis. Unique compound index on (resumeId, jobId).

Routes

- auth.routes.js — Defines /api/auth/register, /api/auth/login, /api/auth/me.
- resume.routes.js — Defines /api/resumes endpoints for upload, list, update, delete, and clear.
- job.routes.js — Defines /api/jobs endpoints for full CRUD.
- analytics.routes.js — Defines /api/analytics for aggregated insight data.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- Install Mongoose by running: npm install mongoose
- Create the database connection module (config/db.js).
- Create Schemas and Models for User, Resume, Job, Skill, and MatchResult.

Create Database connection:

The connection uses a cached Mongoose connection pattern to avoid creating multiple connections in long-running Node.js processes. The connection string is read from the DATABASE_URL environment variable, defaulting to a local MongoDB instance.

Photo

Schemas:

1. User Schema (Users)

- email: String (required, unique) — User's email address, used for login.
- name: String — Display name.
- password: String (required) — Stored as a bcrypt hash.
- avatarUrl: String — Optional profile picture URL.
- timestamps: true — Auto-generates createdAt and updatedAt fields.

2. Resume Schema

- userId: ObjectId (ref: 'User', required) — Links the resume to its owner.
- fileUrl: String (required) — Storage path or URL of the uploaded PDF.
- fileName: String (required) — Original filename.
- parsedText: String — Raw text extracted from the PDF by the parser service.
- resumeVector: [Number] — Semantic embedding generated by the embedder service.
- feedback: String — AI-generated improvement recommendations.
- improvedResumeUrl: String — URL of the AI-enhanced PDF.
- status: Enum (UPLOADED | PARSING | PARSED | EMBEDDING | EMBEDDED | SKILL_EXTRACTED | COMPLETED | FAILED).
- skills: [ObjectId] (ref: 'Skill') — Extracted skills linked to Skill documents.

3. Job Schema

- title: String (required) — Job title.
- company: String — Company name.
- description: String (required) — Full job description text used for matching.
- jobVector: [Number] — Semantic embedding of the job description.
- skills: [ObjectId] (ref: 'Skill') — Skills required by the job.

4. Skill Schema

- name: String (required, unique) — The skill name (e.g., 'React', 'Python').
- category: String — Optional grouping (e.g., 'Frontend', 'Backend', 'DevOps').

5. MatchResult Schema

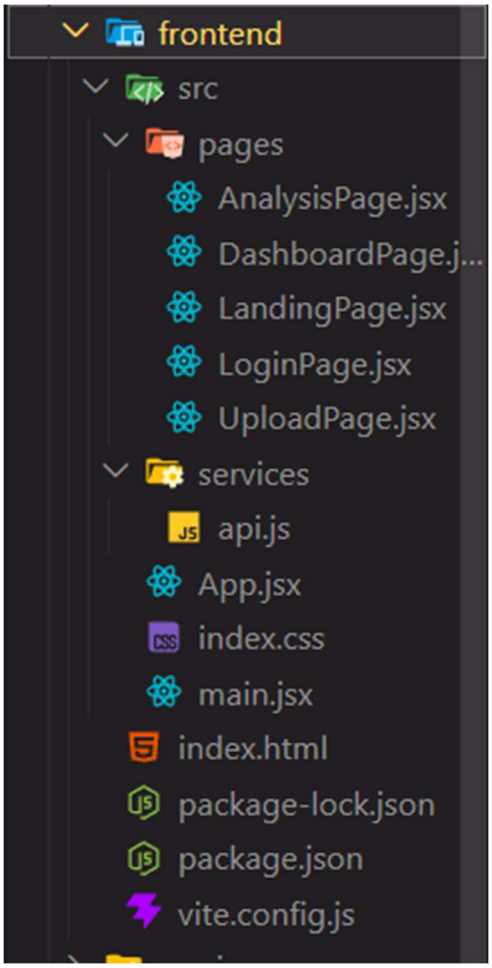
- resumeId: ObjectId (ref: 'Resume', required) — The resume being evaluated.
- jobId: ObjectId (ref: 'Job', required) — The job being matched against.
- score: Number (required) — Raw cosine similarity score.
- matchPercentage: Number (required) — Human-readable percentage match.
- skillGap: [String] — List of skills present in job but missing from resume.
- confidence: Number — Confidence level of the match calculation.
- basis: String — Description of the matching methodology used.
- Unique compound index on (resumeId, jobId) prevents duplicate match records.

FRONT-END DEVELOPMENT:

Frontend Reference Video:

link

Frontend Structure:



Pages (src/pages)

- LandingPage.jsx — The public-facing home page. Features animated sections showcasing AI-Powered Analysis, Instant Matching, Skill Gap Insights, and ATS Optimization. Includes call-to-action buttons directing users to register or login.
- LoginPage.jsx — Authentication page for returning users. Includes client-side form validation and toast notifications for error/success feedback. Navigates to the dashboard on successful login.
- UploadPage.jsx — Resume upload interface featuring react-dropzone for drag-and-drop PDF uploads. Includes a job description entry field and triggers the analysis pipeline on submit.
- AnalysisPage.jsx — Displays the full analysis result: match percentage, skill gap breakdown, AI feedback, and a button to trigger resume enhancement. Shows real-time status updates as the pipeline progresses.
- DashboardPage.jsx — User homepage showing all past resume analyses, match scores visualized with Recharts, aggregated skill gap data, and options to delete individual entries or clear all history.

Services (src/services)

- api.js — Centralised Axios instance with base URL configuration and JWT Authorization header injection for all authenticated requests.

Components

- Reusable UI components shared across pages such as navigation bars, loading spinners, skill badge components, and chart wrappers.
- Framer Motion wrappers for animated page transitions and element entrances.
- React Toastify integration for non-blocking success and error notifications.

Microservices (services/)

- parser/ — Listens on resume_uploaded topic. Downloads and parses the PDF using pdf-parse. Publishes parsed text to resume_parsed topic.
- embedder/ — Listens on resume_parsed topic. Calls Nebius AI to generate semantic vector embeddings. Publishes to resume_embedded topic.
- skill-extractor/ — Listens on resume_embedded topic. Extracts skills from parsed text using NLP / AI. Publishes to skills_extracted topic.
- matcher/ — Listens on skills_extracted topic. Computes cosine similarity between resume and job vectors. Stores MatchResult. Publishes to resume_matched topic.
- feedback/ — Listens on resume_matched topic. Calls AI to generate natural language feedback. Updates the Resume document with the feedback field.
- enhancer/ — Listens on resume_to_enhance topic. Uses Llama-3.3-70B to rewrite and upgrade resume content. Generates a one-page PDF using a custom LaTeX-style engine (pdf.js). Saves the enhanced PDF and updates improvedResumeUrl.

Output ScreenShots:

The following section describes the key screens of the AI Resume Scorer application as implemented in the frontend:

Landing Page:

The public-facing landing page greets visitors with an animated hero section: 'AI Resume Scorer Powered by Intelligence.' Feature cards highlight the four core capabilities: AI-Powered Analysis, Instant Matching, Skill Gap Insights, and ATS Optimization. A prominent 'Get Started Free' CTA button leads to the registration flow.

Login Page:

A clean, centered card with the title 'Welcome Back.' Email and password fields with client-side validation. A 'Log In' button and a link to create a new account. Toast notifications display success or error messages after each submission attempt.

Upload Page:

The upload interface features a large drag-and-drop zone (powered by react-dropzone) that accepts PDF files. Below the upload zone, a text area allows the user to paste or type the job description. A 'Analyze Resume' button triggers the Kafka pipeline and displays a loading indicator while the pipeline processes.

Analysis Page:

Once the pipeline completes, the analysis page displays a circular match percentage gauge (e.g., 78%), a skill gap table listing missing skills highlighted in red, a list of confirmed matching skills in green, and a paragraph of AI-generated feedback. An 'Improve My Resume' button triggers the enhancer service.

Dashboard Page:

The user dashboard shows a summary card for each past resume analysis including the file name, match percentage, and date. A bar chart (Recharts) visualizes match scores over time. A skill frequency chart shows the most commonly missing skills across all analyses. Buttons allow individual entry deletion or bulk history clearing.

Enhanced Resume Download:

After enhancement, a 'Download Enhanced Resume' button appears on the analysis page. The enhanced PDF is formatted in a professional, LaTeX-inspired single-page layout with bold section headers, quantified bullet points, and clean typography using Times-Bold and Italic font pairings.

DEPLOYMENT

Docker/docker-compose

A docker-compose.yml file at the root provides containerized local deployment of all infrastructure dependencies MongoDB, Redis, and Kafka (with Zookeeper). Running docker-compose up starts the entire data and messaging infrastructure, allowing developers to run the application services locally without manual installation.

PM2(Production Process Manager)

An ecosystem.config.js file in the deploy/ folder defines PM2 process configurations for all services: the backend API, the frontend build server, and all six microservices. PM2 manages process restarts, logging, and clustering in production.

Nginx(Reverse Proxy)

An nginx.conf file routes incoming HTTP/HTTPS traffic to the appropriate backend service. It proxies /api/* requests to the Express backend and serves the built React frontend as static files. It also handles SSL termination.

Environment Variables:

DATABASE_URL	MongoDB connection URI (e.g., mongodb://localhost:27017/resume-analyser)
JWT_SECRET	Secret key for signing and verifying JSON Web Tokens
API_PORT	Port for the Express API server (default: 4000)
CORS_ORIGINS	Comma-separated list of allowed frontend origins
REDIS_URL	Redis connection URI (e.g., redis://localhost:6379)
KAFKA_BROKERS	Comma-separated list of Kafka broker addresses
KAFKA_CA_PATH	Path to the Kafka CA certificate file
KAFKA_KEY_PATH	Path to the Kafka client private key
KAFKA_CERT_PATH	Path to the Kafka client certificate

Demo Video Link:

Link

Code Drive Link:

Link